Your First Android App

Android releases and API levels

https://developer.android.com/tools/releases/platforms

API Levels in an App

- Minimum SDK: Device needs at least this API level to install
- Target SDK: API version and highest Android version tested
- Compile SDK: Android OS library version compiled with

```
minSdkVersion <= targetSdkVersion <= compileSdkVersion
```

Anatomy of a basic app project

- Activity
- Resources (layout files, images, audio files, themes, and colors)
- Gradle files

Android app project structure

```
MyApplication
├── app
│   ├── libs
│   └── src
│       ├── androidTest
│       ├── main
│       │   ├── java
│       │   ├── res
│       │   └── AndroidManifest.xml
│       └── test
├── build.gradle
└── gradlew
```

Views

- Views are the user interface building blocks in Android
  - Bounded by a rectangular area on the screen
  - Responsible for drawing and event handling
  - Examples: TextView, ImageView, Button
- Can be grouped to form more complex user interfaces

XML Layouts

You can also edit your layout in XML.

- Android uses XML to specify the layout of user interfaces (including View attributes)

- Each View in XML corresponds to a class in Kotlin that controls how that View functions

XML for a TextView

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"/>
```

Hello World!

Size of a View

- wrap_content

```
android:layout_width="wrap_content"
```

- match_parent

```
android:layout_width="match_parent"
```

- Fixed value (use dp units)

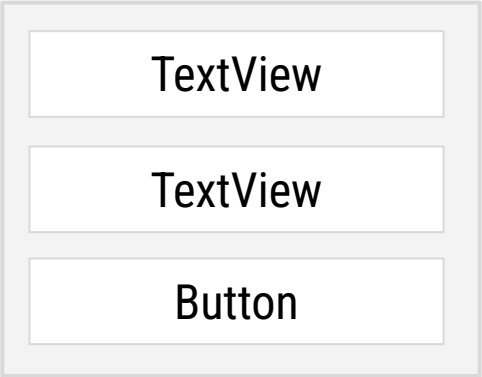
```
android:layout_width="48dp"
```

ViewGroups

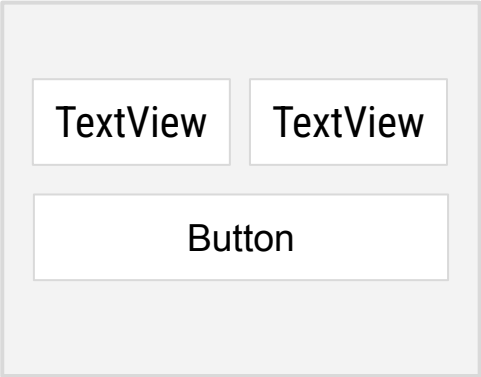
A `ViewGroup` is a container that determines how views are displayed.

FrameLayout

LinearLayout



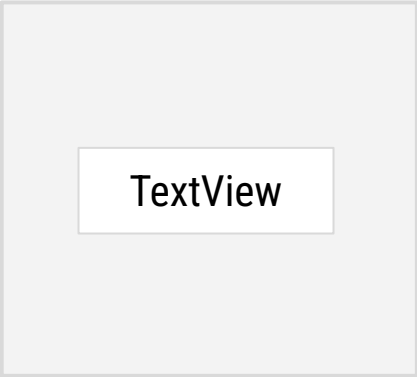
ConstraintLayout



FrameLayout example

A `FrameLayout` generally holds a single child `View`.

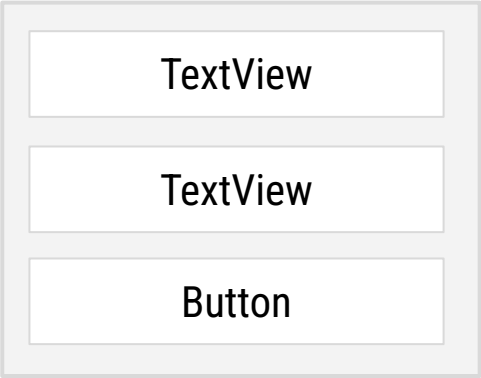
```
<FrameLayout
   android:layout_width="match_parent"
   android:layout_height="match_parent">
   <TextView
       android:layout_width="match_parent"
       android:layout_height="match_parent"
       android:text="Hello World!"/>
</FrameLayout>
```



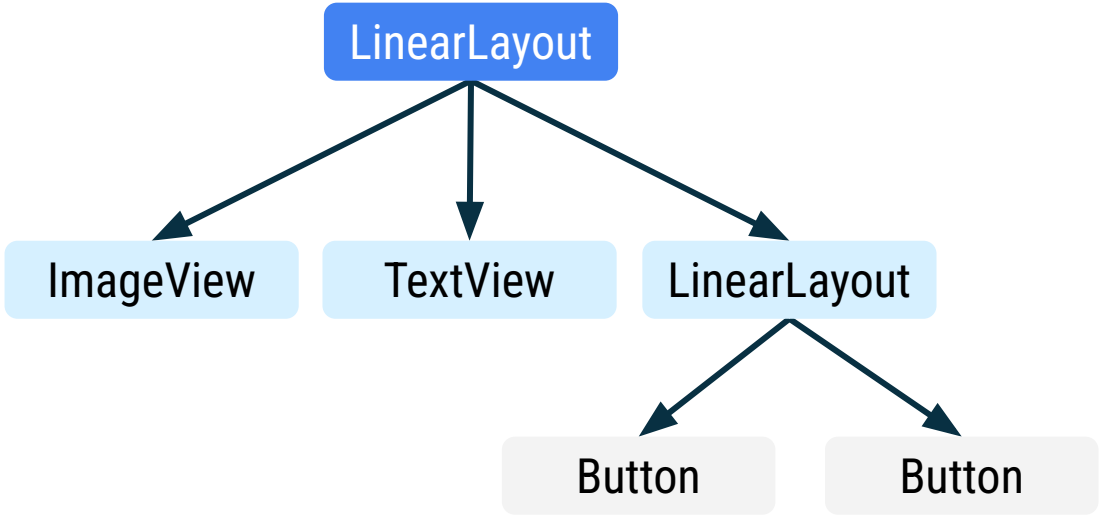
LinearLayout example

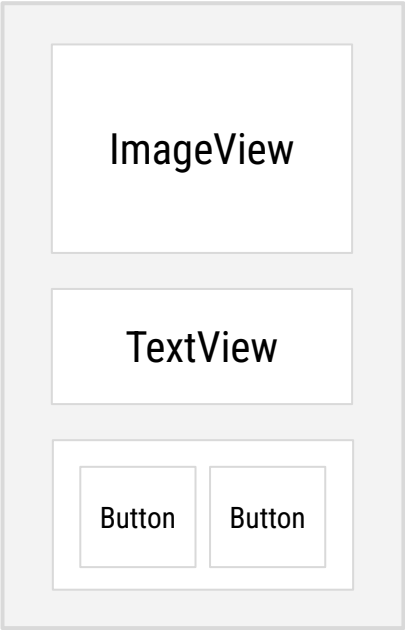
- Aligns child views in a row or column
- Set `android:orientation` to `horizontal` or `vertical`

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <TextView ... />
    <TextView ... />
    <Button ... />
</LinearLayout>
```



View hierarchy





App resources

Static content or additional files that your code uses

- Layout files
- Images
- Audio files
- User interface strings
- App icon

Common resource directories

Add resources to your app by including them in the appropriate resource directory under the parent `res` folder.

```
main
├── java
└── res
    ├── drawable
    ├── layout
    ├── mipmap
    └── values
```

Resource IDs

- Each resource has a resource ID to access it.
- When naming resources, the convention is to use all lowercase with underscores (for example, `activity_main.xml`).
- Android autogenerates a class file named `R.java` with references to all resources in the app.
- Individual items are referenced with:

  `R.<resource_type>.<resource_name>`

Examples: `R.drawable.ic_launcher (res/drawable/ic_launcher.xml)`
`R.layout.activity_main (res/layout/activity_main.xml)`

Resource IDs for views

Individual views can also have resource IDs.

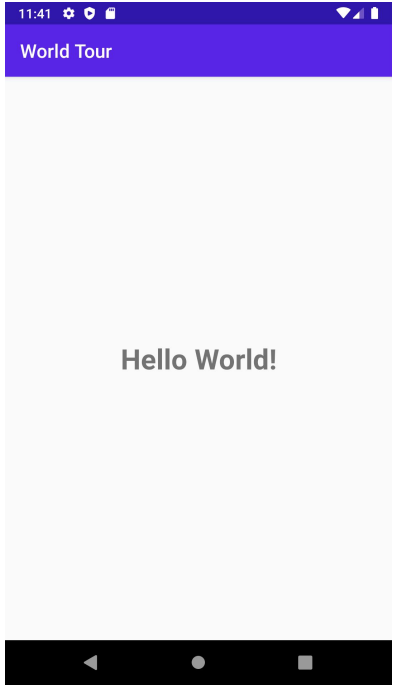
Add the `android:id` attribute to the View in XML. Use `@+id/name` syntax.

```
<TextView
    android:id="@+id/helloTextView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!"/>
```

Within your app, you can now refer to this specific TextView using:

`R.id.helloTextView`

What's an Activity?



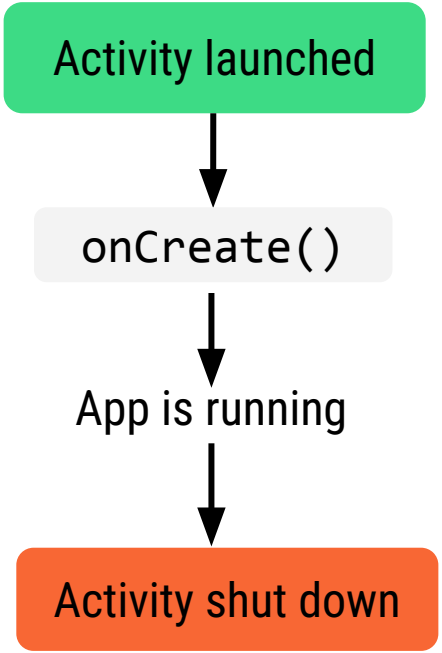
- An Activity is a means for the user to accomplish one main goal.
- An Android app is composed of one or more activities.

MainActivity.kt

```
class MainActivity : AppCompatActivity() {

   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)
       setContentView(R.layout.activity_main)
   }
}
```

How an Activity runs

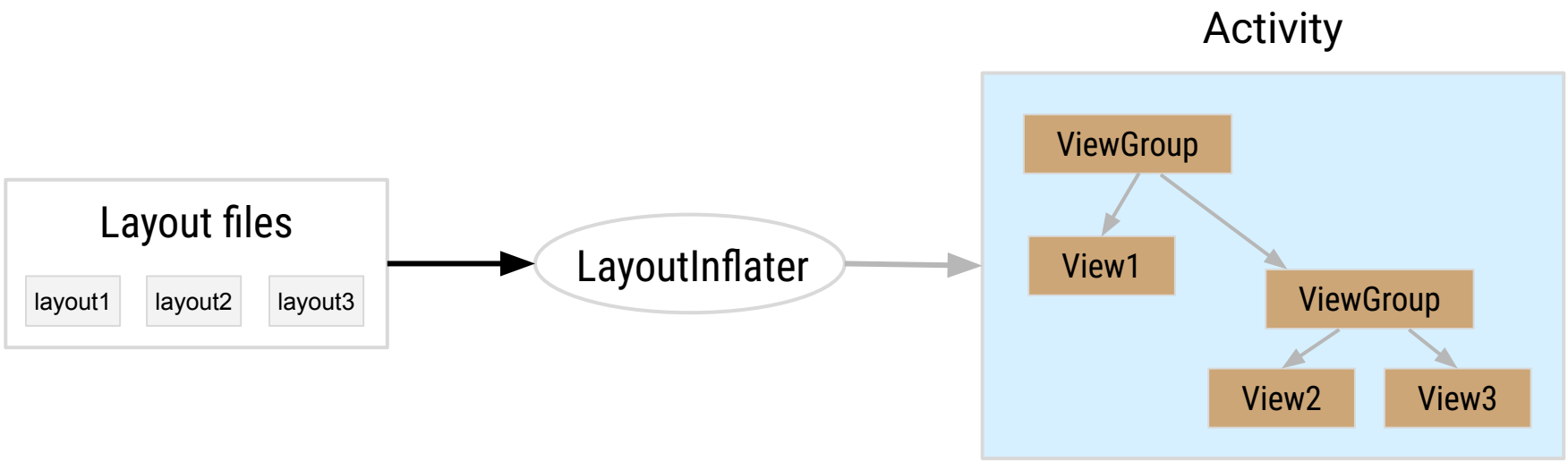


Implement the onCreate() callback

Called when the system creates your Activity

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
}
```

Layout inflation



Modify a View dynamically

Within `MainActivity.kt`:

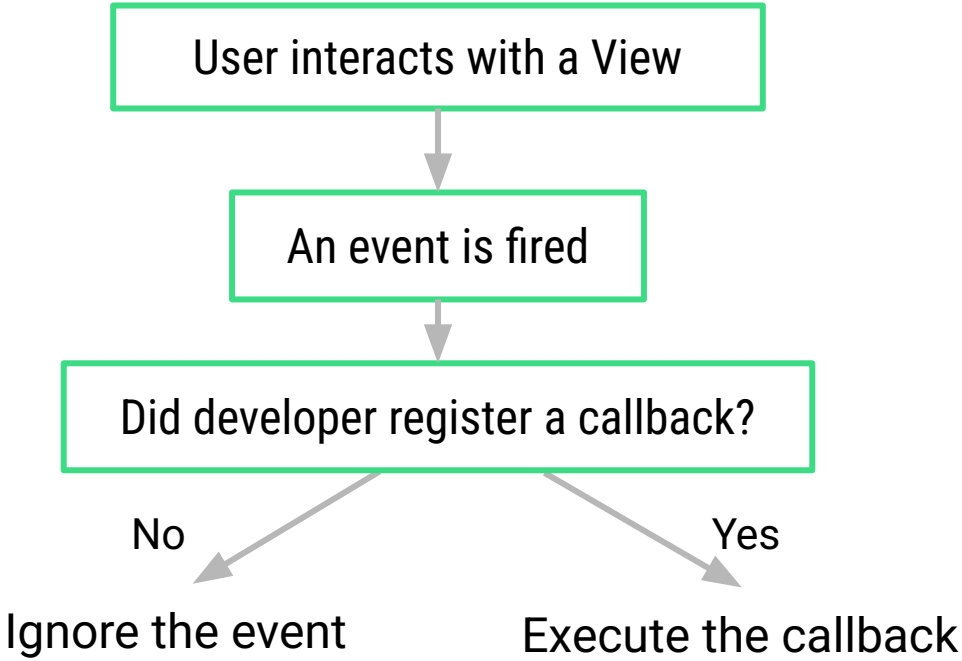
Get a reference to the View in the view hierarchy:

```
val resultTextView: TextView = findViewById(R.id.textView)
```

Change properties or call methods on the View instance:

```
resultTextView.text = "Goodbye!"
```

Set up listeners for specific events



View.OnClickListener

```
class MainActivity : AppCompatActivity(), View.OnClickListener {

    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        val button: Button = findViewById(R.id.button)
        button.setOnClickListener(this)
    }

    override fun onClick(v: View?) {
        TODO("not implemented")
    }
}
```

SAM (single abstract method)

Converts a function into an implementation of an interface

Format: `InterfaceName { lambda body }`

```
val runnable = Runnable { println("Hi there") }
```

is equivalent to

```
val runnable = (object: Runnable {
    override fun run() {
        println("Hi there")
    }
})
```

View.OnClickListener as a SAM

A more concise way to declare a click listener

```
class MainActivity : AppCompatActivity() {

   override fun onCreate(savedInstanceState: Bundle?) {
       ...

       val button: Button = findViewById(R.id.button)
       button.setOnClickListener({ view -> /* do something*/ })
   }
}
```

Late initialization

```
class Student(val id: String) {

    lateinit var records: HashSet<Any>

    init {
        // retrieve records given an id
    }
}
```

Lateinit example in Activity

```
class MainActivity : AppCompatActivity() {

    lateinit var result: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        ...
        result = findViewById(R.id.result_text_view)
    }
}
```

What is Gradle?

- Builds automation system
- Manages the build cycle via a series of tasks (for example, compiles Kotlin sources, runs tests, installs app to device)
- Determines the proper order of tasks to run
- Manages dependencies between projects and third-party libraries

Gradle build file

- Declare plugins
- Define Android properties
- Handle dependencies
- Connect to repositories

Plugins

Provide libraries and infrastructure needed by your app

```
apply plugin: 'com.android.application'
apply plugin: 'kotlin-android'
apply plugin: 'kotlin-android-extensions'
```

Android configuration

```
android {
   compileSdkVersion 30
   buildToolsVersion "30.0.2"

   defaultConfig {
       applicationId "com.example.sample"
       minSdkVersion 19
       targetSdkVersion 30
   }
}
```

Dependencies

```
dependencies {
   implementation "org.jetbrains.kotlin:kotlin-stdlib-jdk7:$kotlin_version"
   implementation 'androidx.core:core-ktx:1.3.2'
   implementation 'androidx.appcompat:appcompat:1.2.0'
   implementation 'com.google.android.material:material:1.2.1'
   ...
}
```

Repositories

```
repositories {
    google()
    jcenter()
    maven {
        url "https://maven.example.com"
    }
}
```

Common Gradle tasks

- Clean
- Tasks
- InstallDebug

Accessibility

- Refers to improving the design and functionality of your app to make it easier for more people, including those with disabilities, to use
- Making your app more accessible leads to an overall better user experience and benefits all your users

Make apps more accessible

- Increase text visibility with foreground and background color contrast ratio:
  - At least 4.5:1 for small text against the background
  - At least 3.0:1 for large text against the background
- Use large, simple controls
  - Touch target size should be at least 48dp x 48dp
- Describe each UI element
  - Set content description on images and controls

Add content labels

- Set `contentDescription` attribute → read aloud by screen reader

```
<ImageView
    ...
    android:contentDescription="@string/stop_sign" />
```

- Text in TextView already provided to accessibility services, no additional label needed

No content label needed

- For graphical elements that are purely for decorative purposes, you can set

  ```
  android:importantForAccessibility="no"
  ```

- Removing unnecessary announcements is better for the user

TalkBack

- Google screen reader included on Android devices
- Provides spoken feedback so you don’t have to look at the screen to use your device
- Lets you navigate the device using gestures
- Includes braille keyboard for Unified English Braille